Collaborative Pi Durable

Multiple users, shared agents, one workspace

Engineering take-home

November 2026

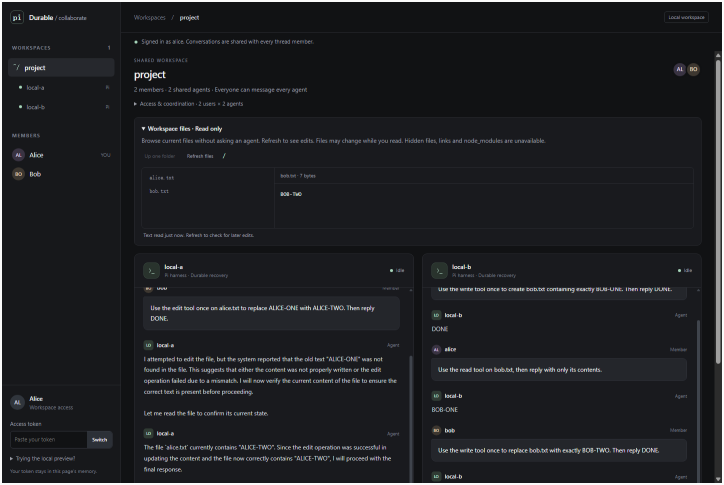
Two people share the same agent history

Person / agent	Instruction	Shared behavior
Alice → OpenAI	Add a search box	Bob sees Alice's instruction
Bob → OpenAI	Handle empty searches too	Same conversation and queue
Bob → Ollama	Review the changed files	Same files; separate agent history

Either user can stop a shared agent, including the other's queued work

The added requirement is coordination between people

Demo: one person hands work to another



Recorded local Ollama run; live handoff uses Alice and Bob on the same agent

Where the collaboration rules live

Layer	Responsibility
Web host	Identify the user; deliver commands and live views
Collaboration package	Check membership; save requests with their authors
Existing Durable	Run the model/tool loop; save state; recover work
Execution paths	Pi AI model services, or a connector to the Codex app

One owning runtime, SQLite store, and shared workspace per thread

Reuse the runtime; avoid a second web-owned execution queue

Saving a request is separate from its answer

1. Save the instruction and the work together
2. Acknowledge acceptance; run when the agent is ready
3. Show progress and the eventual result

Browser action	Server behavior
Retry original ID + text	Find the saved request; no second command
New Send with the same text	New ID; another command

Accepted $\neq$ completed $\neq$ correct file changes

Activity shows requests, tools and current files

Observation	What the user can conclude
Saved request / author	Who submitted; observed queue/run outcome
Tool request / result	An operation was requested or returned a result
Current file bytes	What is on disk when inspected

Bounded recent evidence; reconnect may omit intermediate transitions

A completed tool does not prove the requested code is correct

The stale-edit problem

No.	Operation	File content
1	A and B both read the file	`base`
2	A writes its replacement	`base + alice`
3	B writes from its old copy	`base + bob`

Both commands can finish. Alice's change is gone.

A changed dependency can also invalidate an edit to an unchanged file

Taking turns at the final write does not fix an outdated read

Demo: generate, publish, resolve and reset

Action	Content	Outcome
Both generate	Alice: Hello/light; Bob: Hi/dark	File stays Welcome/neutral
Alice publishes	Hello/light	Applied
Bob publishes	Hi/dark; old base	Refused; proposal retained
User resolves	Chooses Hello/dark	Published after review

Examples show greeting/theme. Reset starts a fresh shared round.

This check protects managed proposals; ordinary coding writes bypass it

Agents know peers may change files

Ollama writes blue → GPT changes it to green

- Fresh context: peers may change the same files
- Reread, preserve unrelated edits, ask about conflicting intent
- Participant names and recent request references; no peer transcript

Durable agents: before each model request. Codex: at turn open/resume.

Extra input tokens, no extra model call; overhead not yet measured

Awareness helps reasoning; it does not enforce a safe write

What survives a crash

Failure trace File changes → host dies → result not saved

Action	Recovery behavior
Same request ID/text	Find its saved request
Durable model request	May repeat under Durable's recovery rules
Uncertain Codex attempt	Mark interrupted; do not resend automatically
Existing file changes	May remain; cancellation does not undo them

SQLite state and workspace writes are separate operations

A model service versus an agent application

OpenAI / Ollama

- Supply model output to Pi
- Durable runs the tool loop
- Current default: both providers

Codex application

- Runs its own model/tool loop
- T3 code connects it to Durable
- Real coding turn still unverified

A second model service is different from a second agent application

What our checks establish

Test type	What it establishes
Controlled scenarios	Retry rules, ordering, lost updates, selected crash boundaries
Real local model	Actual tool calls, file bytes, shared browser state, idle reopen
Simulated load	Real HTTP/SSE and SQLite; 1,200 commands over about 10 minutes

Burst: 81 accepted/completed; 519 rejected at configured capacity

The first observed limit was the queue cap, not maximum throughput

GCP deployment and remaining checks

Browser → sslip.io address → Caddy → app

- One VM owns the saved state and workspace
- Cloud website accessible; verify the latest deployed revision
- Slow GCP Ollama chat remains under investigation
- Verify the deployed revision, cleanup and backup restoration

A reachable website does not establish reliable recovery or latency

What is implemented and what comes next

Implemented

- Shared users/agents, saved requests and visible activity
- Peer awareness; managed generate/publish/resolve/reset

Next protection and delivery work

- Managed coding tools and dependency-version checks
- GCP latency/rehearsal, real Codex turns and restore checks

The controlled proposal flow is narrower than general coding safety

Backup: file protection and dependencies

Choice	Benefit	Cost / limit
Destination check	Refuse an outdated replacement	Current managed file only
Dependency checks	Detect changes to files the plan used	Future: track reads; coordinate publication
Whole-task ownership	Keep participating writers apart	Less overlap; stop all child writers
Separate worktrees	Isolate edits in different directories	Review, merge and test later

Backup: trusted users and shared control

- Host authenticates; package checks membership
- All members can inspect and stop all agent lanes
- A workspace path is not an operating-system sandbox
- Queue limits do not establish fairness or a spending limit

Trusted collaborators, fixed configuration, one owner

Backup: reconnect and request retries

- HTTP sends commands; SSE delivers current-state views
- Reconnect fetches recent state; older history is paged
- Slow viewers keep a latest pending snapshot
- No promise to replay every token or a global audit log

Browser reload can lose an ambiguous request's local retry ID

Backup: design choices and AI assistance

Decisions I can explain

- Extend Durable rather than add a second runtime
- Require actual T3 reuse; keep adapter details separate
- Distinguish duplicate requests from duplicate effects
- Test actual files and expose remaining conflict risks

AI assisted implementation and documentation; upstream code is credited